SEAGAS - Work Report: Database, Admin Accounts and Login

Carl (Soh Way Miin) - Backend Developer | 12 October 2026
COS40005 Computing Technology Project A | Group 10 | POC004

1. Summary

Today I checked our database against the Sprint 1 report, fixed what did not match, and changed how accounts and sign-in work. Students still register themselves, but advisor and admin accounts can now only be created by an admin, and every new account must use the NewInti email domain for its role. Everyone signs in on one login page, and the system opens the student, advisor or admin pages based on the role stored in the database. A new admin interface manages accounts. All work is in 6 commits, pushed to `dev` (now at `50b0688`).

Commit	Type	What it does
8a2993d	fix(database)	Align the schema with the Sprint 1 report: skills_library subcategory and source, fix the advisor progress view, add the missing ORM model, and add a migration file for older databases.
017f6f4	feat(admin)	Only students can self-register; admins create advisor and admin accounts. Admin API, admin pages and create_admin.py.
94c2eee	fix(database)	Load the recommendations library (298 resources) on every backend start, with the unique constraint it depends on.
065fa1d	feat(auth)	Separate student, advisor and admin login pages (replaced by the next commit).
4803bf7	feat(auth)	One login page; the role in the database decides which portal opens. Student, advisor and admin code is fully separated.
50b0688	feat(auth)	New accounts must use @student / @advisor / @admin.newinti.edu.my.

2. Database Check Against the Sprint 1 Report

The concern was that the database was missing advisor data and student assessment records. The check found the following:

Item	Finding	Action
Assessment records	Already complete. `assessments` stores every run (overall + 7 dimension scores) and `skill_match_results` stores the per-skill result. History is kept, nothing is overwritten.	None needed
Advisor data	Advisors exist only as `users.role = 'advisor'`. The report plans advisor notes for a later sprint.	Kept as in the report; accounts are now created by an admin
Advisor cohort progress	Bug: `v_student_progress` read `student_recommendations`, which nothing writes to, so the advisor progress endpoint was always empty.	View now reads `student_resource_progress`, the table progress.py writes
Skills library fields	Report Figure 44 shows subcategory and source, but the table had neither. The CSV already had them; the seed generator dropped them.	Columns added; seed regenerated (138 skills, 32 v1)
ORM models	`student_resource_progress` had no model.	Model added; models.py now covers all 15 tables in the report
Recommendations library	298 resources existed only in my local database. The seed was not in start_backend.bat and its unique constraint was added by hand.	See section 3.2

Item	Finding	Action
Test account	`user@example.com` (name 'string'), left over from Swagger testing, with no profile.	Deleted from my local database

A full structural comparison with a database freshly built from `schema.sql` found every column, constraint and view identical after the fixes.

3. Changes

3.1 Schema aligned with the report

- `schema.sql`: added `subcategory` and `source` to `skills_library`, and moved `v_student_progress` onto `student_resource_progress`.
- `data/skills/build_skills_seed.py` now writes subcategory and source; `seed_skills_library_v2.sql` regenerated.
- New `database/migrate_existing_db.sql`: brings databases created from an older schema up to date. It is idempotent and `start_backend.bat` runs it on every start, so teammates do not need to recreate their database.

3.2 Recommendations library for every team member

- `start_backend.bat` now loads `seed_recommendations_library_final.sql` after the skills and job role seeds. A new database gets all 298 resources.
- The unique constraint `uq_resource_skill (skill_id, resource_title)` is now in `schema.sql` and the migration, so loading the seed on every start never adds duplicates.
- If a teammate's database already has duplicates from loading the seed twice, the migration keeps the oldest copy of each resource and moves students' progress rows onto it before deleting the rest.

3.3 Account management

- `POST /api/auth/register` only creates students. Any other role is rejected (403), and the Register page no longer offers an Advisor option.
- New admin-only router `/api/admin`: account counts, paged user list with search and role filter, create advisor or admin, rename, activate / deactivate (not your own account), reset password, and read-only skills library and job role lists.
- Deactivated accounts cannot sign in, and a token issued before deactivation stops working.
- `backend/create_admin.py` creates the first admin interactively (email, name, hidden password). `start_backend.bat` offers to run it when there is no admin. No usernames or passwords are stored in the code.

3.4 Admin interface

- **User Management**: counts per role, search, role filter, 50 accounts per page, Add user, Reset password, Activate / Deactivate.
- **Skills & Roles**: the 138 skills (category, subcategory, aliases, source, roles using it) and the 6 job role templates. Read-only, because the library is maintained in `skills_library.csv`. Matches the Administrator wireframe (report Figure 39).

3.5 One login page, fully separated portals

- Everyone signs in on `/login` with email and password only. The backend returns the account's role from `users.role` and the app opens the student, advisor or admin portal. Users never choose a role. "Create an account" stays and only creates students.
- The role is confirmed with `/api/auth/me` every time the app loads; only the token is kept in the browser, so editing browser storage cannot open another role's pages.
- Each role's pages are a separate, lazy-loaded portal (`frontend/src/portals`), and the advisor and admin API calls moved to `api/advisor.js` and `api/admin.js`. A student's browser never downloads advisor or admin pages or their API addresses (checked in the production build).

- Backend role checks on every advisor and admin endpoint are unchanged, so data stays protected even if someone calls the API directly.

3.6 Email format for new accounts

Role	Email must end with	Created by
Student	`@student.newinti.edu.my`	The student, on the Register page
Advisor	`@advisor.newinti.edu.my`	An admin, in User Management
Admin	`@admin.newinti.edu.my`	An admin, or create_admin.py for the first one

- The role still comes from the database. The domain rule only guarantees that a new account's email matches its role, so a student cannot register an advisor-looking address.
- The rule lives in one place (`ROLE_EMAIL_DOMAINS` in `routers/auth.py`). Emails are stored in lower case and matched case-insensitively.
- Existing accounts are not affected: the 5,000 synthetic students keep `@synthetic.example.com` and still sign in.

4. Changes to My Local Database (not in git)

Change	Result
Deleted the Swagger test account	`user@example.com` removed. Students and profiles now match: 5,000 each.
Renamed the admin account	Aaron Tee Yong Xuan: `admin01@gmail.com` to `aaron@admin.newinti.edu.my` (password unchanged).
Renamed the advisor account	Dr. Shan: `advisor@seagas.com` to `shanshan@advisor.newinti.edu.my` (password unchanged).
Final state	1 admin, 1 advisor, 5,000 synthetic students; 15 tables and 3 views; 138 skills; 6 job roles with 152 skill links; 298 recommendation resources.

These two accounts exist only in my database. Each teammate creates their own admin on first start, and their own advisors from User Management.

5. Testing

Backend tests ran against temporary copies of the database, which were deleted afterwards; the real `seagas_db` was not used for testing.

Area	Result	What was checked
Schema and migration	Pass	New database and a copy of an old one; migration run twice; advisor progress view counts completed / pending correctly.
Recommendations seed	Pass	New database: 298 rows after two starts. Simulated old database with 596 rows (seed loaded twice): cleaned to 298, constraint added, students' progress kept.
Admin and account API	29 / 29	Self-registering as advisor/admin blocked; admin creates advisor; unknown email cannot sign in; deactivate blocks sign-in and old tokens; reset password; students and advisors get 403 on admin API; search, filter, paging.
Email domain rules	15 / 15	Wrong and look-alike domains rejected for each role; correct domains accepted; case-insensitive duplicates rejected; existing synthetic and old accounts still sign in.
Frontend	Builds	Production build and lint pass. The bundle check confirmed admin and advisor code is only in their own portal files. Admin User Management was confirmed working in the browser; the single login flow still needs a full manual test.

6. What Teammates Need to Do

- `git checkout dev` and `git pull`, then run `backend\start_backend.bat`.
- The database updates itself: new columns, constraint, advisor progress view and 298 recommendation resources.
- When asked, create an admin account. The email must end with `@admin.newinti.edu.my`.
- Sign in as admin, open User Management and create advisors with `@advisor.newinti.edu.my` emails.
- The synthetic test login `student0001@synthetic.example.com` still works.

Note: these 6 commits were pushed straight to dev without a pull request, so please review the changes listed in section 3.

7. Open Items

Item	Detail	Suggested owner
JD analysis placeholder	15 job descriptions are marked `complete` but `jd_extracted_skills` is empty: `POST /api/jobs/descriptions/{id}/analyse` still marks them complete without calling NLP. Yong Hin's `nlp/extractor.py` is now on dev and can be connected.	Carl + Yong Hin (S2)
Advisor 'Profiles Complete'	Counts all 5,000 profiles; `is_complete` says 3,823 are complete and 1,177 are not.	Carl
Advisor notes and assignment	No table for advisor notes or advisor-student assignment yet. The report lists Student Detail and notes as planned for a later sprint.	Later sprint
Skills library editing	The admin Skills & Roles page is read-only; changes still go through the CSV and seed files.	Later sprint
Manual UI test	Walk through sign-in for each role, refresh, sign-out and the Register page in the browser.	Ren Hang / team